Toollery: Scaling LLM Agents to Hundreds of Tools

Anonymous ACL submission

Abstract

LLM agents increasingly rely on external skills, from high-level procedures to atomic tools and APIs, but selecting the right skill becomes difficult as skill libraries grow. Full-library prompting scales poorly in tokens and latency, while direct retrieval over skill metadata often misses the way users express their intents. We introduce **Toollery**, a training-free candidate-compression framework that enriches skill and tool metadata with generated user-intent queries before final LLM decision making. Toollery generates user-like queries for each skill, attaches them to the searchable skill document, retrieves a compact candidate set with a standard retriever, and forwards only the top candidates to the final LLM. On SkillBench-compatible routing over roughly 79K-skill pools, Toollery with BM25 reaches .920 Hit@1, .942 Recall@10, .968 Recall@50, .942 MRR@10, .880 nDCG@10, and .853 Full-Coverage@10. On BFCL-live atomic-skill retrieval over a complete 457-tool pool, Toollery with RAGAnything reaches .776 Hit@1 and .980 Recall@10. These results show that generated user intents are a strong transferable retrieval signal, and that the same Toollery representation can be paired with different retrieval backends.

1 Introduction

LLM agents increasingly rely on external capabilities, from high-level procedural skills to atomic tools and APIs (Schick et al., 2023; Qin et al., 2023; Patil et al., 2024; Yao et al., 2022). As public ecosystems already aggregate large collections of agent skills[1] /tools, selecting *which* capability to expose becomes a bottleneck for both effectiveness and efficiency (Lin et al., 2025; Mo et al., 2025; Xu et al., 2025).

[1] https://github.com/VoltAgent/awesome-openclaw-skills; https://github.com/openclaw/clawhub.

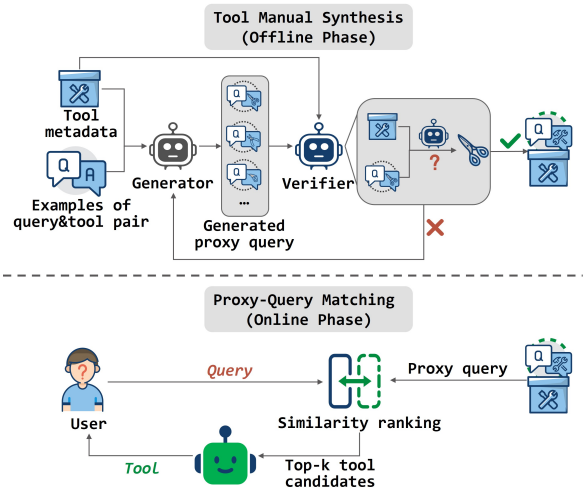

Figure 1: **Overview of Toollery.** Toollery scales skill selection by moving part of intent matching offline. For each skill specification, a generator produces user-intent queries that are attached to the searchable skill or tool metadata. At inference time, Toollery retrieves a compact candidate skill set from these query-augmented documents and passes only these candidates to the final LLM for decision making. Atomic tools and APIs are treated as atomic skills.

A straightforward solution is to place all skill/tool specifications in the prompt and let the LLM choose. This approach preserves the full decision context, but it scales poorly with library size: prompt length, latency, and distractor interference all increase as more skills/tools are added. Even when the full library fits, long candidate lists can distract the model and hurt selection accuracy (Paramanayakam et al., 2025). Practical agents that expose operating-system actions, atomic tools, MCP services, and skills therefore need a way to narrow the candidate set before LLM inference (Wu et al., 2024; Amazon Web Services, 2024; Mo et al., 2025; Xu et al., 2025).

Another common strategy is to retrieve candidates from names, descriptions, or documentation (Agarwal et al., 2025; Lin et al., 2025). However,

users describe goals and constraints, while specifications describe implementation details, inputs, and developer-facing functionality. This mismatch is especially problematic for heterogeneous skill libraries.

We frame scalable skill selection as **intent-level retrieval**: instead of asking only what a skill says, we ask what user requests should select it. Toollery moves this matching problem partly offline by converting specifications into user-intent queries; online retrieval then operates over these intent representations before passing a compact candidate set to the final LLM.

We introduce **Toollery**, a training-free framework that generates user-like intents for each skill, attaches them to searchable metadata, and retrieves only the top candidate skills for final LLM selection (Figure 1). This separates library-scale matching from schema-aware final decision making.

Our experiments show that generated intent queries are effective at realistic library scales. On SkillBench-compatible routing over roughly 79K-skill pools (Zheng et al., 2026), Toollery with BM25 achieves .920 Hit@1, .942 Recall@10, .968 Recall@50, .942 MRR@10, .880 nDCG@10, and .853 FullCoverage@10, substantially improving over raw metadata retrieval. On BFCL-live atomic-skill retrieval over a complete 457-tool pool, Toollery with RAGAnything reaches .776 Hit@1 and .980 Recall@10. Ablations show that retrieving generated queries alone can be useful, but the main Toollery representation is the combination of original metadata and generated user-intent queries.

This paper makes the following contributions:

- We formulate scalable skill selection as intent-level retrieval, where tools are treated as atomic skills and skill specifications are indexed through generated user intents.
- We present Toollery, a training-free candidate-compression framework that converts skill specifications into user-intent queries, appends those queries to searchable metadata, and retrieves compact candidate sets before final LLM decision making.
- We evaluate Toollery on large-scale skill routing and atomic-tool retrieval settings, showing strong performance over roughly 79K-skill pools and 457-tool API-style pools, with generated-query-only and retrieval-backend ablations.

2 Related Work

Prior work improves LLM tool use by training or evaluating models that invoke external tools once relevant tools are available in context or training data (Yao et al., 2022; Schick et al., 2023; Patil et al., 2024; Qin et al., 2023; Li et al., 2023; Patil et al., 2025). Toollery addresses the complementary deployment problem of selecting a compact candidate set from a large skill library before final LLM decision making.

Recent retrieval and routing methods reduce the action space before tool selection (Agarwal et al., 2025; Lin et al., 2025; Zheng et al., 2026; Gan and Sun, 2025; Paramanayakam et al., 2025). MassTool learns search-based tool retrieval signals, SkillRouter retrieves and reranks skills at scale, and RAG-MCP-style systems retrieve tool documentation to reduce prompt bloat. Toollery is complementary: it trains no router, and expands each searchable skill document with generated user intents so standard retrievers can match user phrasing to developer-facing metadata.

This design is also related to synthetic query/document expansion for reducing vocabulary mismatch (Nogueira et al., 2019). We adapt it to agent skill selection, where each generated query must map back to a selectable skill, and evaluate on recent large-pool skill and tool benchmarks (Zheng et al., 2026; Patil et al., 2024; Wang et al., 2025; Mo et al., 2025; Xu et al., 2025).

3 The Toollery Design

Toollery separates large-library search from final LLM decision making. Figure 1 shows the overall pipeline: Toollery first generates user-intent queries for each skill, appends them to the original searchable metadata, and then uses a standard retriever at inference time to retrieve a compact candidate set for the final LLM. Throughout the paper, we use *skill* as the general unit of selection. A skill may be a high-level procedural capability, while a tool or API function is treated as an atomic skill.

3.1 Problem Formulation

Let $\mathcal{A} = \{a_1, \ldots, a_M\}$ denote a library of available skills. Each skill a_i has a specification s_i, which may include a name, natural-language description, argument schema, examples, usage constraints, documentation, or longer skill instructions. Given a user request x, the goal is to construct a

small candidate set

$$\mathcal{C}_k(x) \subset \mathcal{A}, \qquad |\mathcal{C}_k(x)| = k \ll M, \tag{1}$$

that contains the skill or skills needed to satisfy the request. A downstream LLM then receives x and compact specifications for only $\mathcal{C}_k(x)$, and produces the final skill decision and any required arguments.

The main difficulty is that user requests and skill specifications are written for different purposes. Users describe goals, while specifications describe capabilities, parameters, and constraints. Toollery therefore does not use skill metadata as the only searchable object. Instead, it expands each searchable skill document with generated user-like queries that describe when the skill should be selected.

3.2 Generated Query Construction

For each skill a_i, a generator model G produces N candidate intent queries from the specification:

$$Q_i = G(s_i; N, \pi), \tag{2}$$

where π is a prompt that asks the generator to write plausible user requests that would require a_i. For atomic skills, the generator observes the tool or API name, description, schema, return information when available, and examples. For higher-level skills, the generator observes the capability description, triggering conditions, expected inputs, and constraints.

Toollery then constructs a query-augmented searchable document

$$d_i = \mathrm{concat}(s_i, Q_i), \tag{3}$$

where s_i is the original skill or tool metadata and Q_i is the generated query set. This is the main Toollery representation used in our experiments: retrieval sees both developer-facing metadata and user-facing generated intents.

Generated queries can also be used without metadata as an ablation. Because such a generated-query-only index lacks the original skill description, we optionally filter generated queries with a source-skill recovery check. For each generated query $q \in Q_i$, Toollery constructs a verification pool

$$\mathcal{P}_{i,q} = \{a_i\} \cup \mathcal{D}_{i,q}, \tag{4}$$

where $\mathcal{D}_{i,q} \subset \mathcal{A} \setminus \{a_i\}$ is a set of random distractor skills. The verifier model H receives q and compact specifications for the skills in $\mathcal{P}_{i,q}$, then selects the best matching skill or abstains:

$$\hat{a} = H(q, \{s_j : a_j \in \mathcal{P}_{i,q}\}). \tag{5}$$

Toollery retains q only when the verifier recovers the source skill, $\hat{a} = a_i$. The retained intent set for skill a_i is

$$V_i = \{q \in Q_i : H(q, \{s_j : a_j \in \mathcal{P}_{i,q}\}) = a_i\}. \tag{6}$$

The generated-query-only ablation index is then

$$\mathcal{I} = \{(v, a_i) : v \in V_i,\ a_i \in \mathcal{A}\}. \tag{7}$$

Each index entry stores the retained intent text, its retrieval representation, and the identifier of its source skill. The recovery check tests whether a generated query remains discriminative under competition; because it is offline, its cost is amortized and only affected skills need updates when the library changes.

3.3 Retrieval

At inference time, Toollery applies a retrieval function over the query-augmented document collection $\mathcal{D} = \{(d_i, a_i)\}_{i=1}^{M}$. This retrieval function is not tied to a specific implementation: it can be instantiated with sparse lexical retrieval such as BM25 or a RAG-style neural retriever. Given a user request x, the retriever returns the top-k skills and their scores:

$$\mathcal{C}_k(x) = \mathrm{TopK}_{(d_i,a_i)\in\mathcal{D}}\, f(x, d_i). \tag{8}$$

The key design choice is therefore not the scoring function itself, but the retrieval content: Toollery retrieves over skill or tool metadata enriched with generated user-intent queries rather than over raw metadata alone.

For the generated-query-only ablation, retrieval is applied over $\mathcal{I}$ instead of $\mathcal{D}$. Since multiple retrieved queries may point to the same skill, we aggregate query-level evidence by averaging the strongest retrieved query scores per skill and adding a small evidence-count bonus. This converts many query-level matches into a ranked list of unique skills; exact aggregation parameters are listed in Appendix B.

3.4 Candidate Compression and Final Selection

For each selected skill, Toollery instantiates a compact specification containing the fields required by the final decision interface, such as the skill

name, description, schema, and necessary usage constraints. The final LLM receives the original user request and these compact specifications, but not the generated intent queries. Retrieval therefore narrows the action space; the LLM remains responsible for resolving residual ambiguity, selecting the final skill, and filling arguments.

This design changes the online cost from full-library prompting to subset prompting. If L_s is the average token length of a compact skill specification, all-candidate prompting sends approximately $O(M \cdot L_s)$ skill-definition tokens to the LLM. Toollery sends approximately $O(k \cdot L_s)$ tokens, where $k \ll M$ can remain fixed as the library grows. The query-augmented retrieval index grows offline with the library, but online retrieval is substantially cheaper than expanding the LLM context over all candidates.

4 Experiments

We evaluate Toollery as a candidate-compression layer for skill selection, asking whether query-augmented retrieval recovers correct skills at large scale, transfers to atomic API-style tools, and improves final LLM selection as candidate pools grow.

4.1 Experimental Setup

Benchmarks. We use two retrieval-focused tracks and one end-to-end selection track. The first retrieval track follows SkillBench/SkillRouter-style skill routing, where each user request is evaluated against roughly 79K candidate skills. The second retrieval track uses BFCL non-live and live multiple instances and evaluates each request against the complete candidate pool, treating each tool/API as an atomic skill. The end-to-end track evaluates final LLM selection after candidate compression on BFCL-V4 non-live and live multiple settings under increasing candidate-pool sizes.

Settings and baselines. We compare *Raw metadata*, *Generated query only*, and *Toollery*, which retrieves over metadata plus generated intents. Each setting uses BM25 or RAGAnything-style retrieval. Our dense implementation produced redundant quality values with RAGAnything, so the main tables omit separate dense rows. The SkillRouter row is an external reference from the SkillRouter paper, not a latency-controlled local rerun.

Metrics. For retrieval, we report Hit@1, Recall@10/50, MRR@10, nDCG@10, and FullCoverage@10; for end-to-end experiments, we report final selection accuracy after the LLM chooses from the retrieved subset.

Efficiency. Tables 1, 2, and 3 include online retrieval latency as Avg ms/query, excluding offline intent generation, verification, and index construction. Appendix B reports reproducibility details and offline construction cost; with DeepSeek-V4-Pro pricing, query construction costs about $0.22 per 100 skills.

4.2 Skill Routing over Large Skill Pools

Table 1 reports SkillBench-compatible retrieval averaged over the easy and hard splits. Toollery with BM25 reaches .920 Hit@1, .942 Recall@10, .968 Recall@50, .942 MRR@10, .880 nDCG@10, and .853 FullCoverage@10 over roughly 79K-skill pools, substantially improving over our locally run raw-metadata baselines. The SkillRouter row is contextual only, not a controlled head-to-head comparison.

Generated-query-only retrieval is also strong, especially with RAGAnything, showing that generated intents are useful by themselves. Toollery BM25 is stronger on most metrics because generated queries and original metadata provide complementary evidence.

4.3 Atomic-Skill Retrieval on BFCL

The BFCL tracks test transfer to atomic API-style selection. Table 2 evaluates BFCL-live multiple against all 457 live tools; Table 3 repeats the protocol on 443 non-live tools. Toollery with RAGAnything reaches .776 Hit@1 and .980 Recall@10 in the live setting, and .845 Hit@1 with perfect Recall@10/50 in the non-live setting.

Generated-query-only retrieval is weaker here because API schemas are short and specific; generated intents work best when attached to the original tool document. In live top-3 efficiency, Toollery with RAGAnything uses 653.5 prompt tokens, takes 139.7 ms/query, and reaches .776 Hit@1, while BM25 is faster at 5.0 ms/query and reaches .731 Hit@1.

4.4 End-to-End Selection under Candidate-Pool Growth

We evaluate end-to-end atomic-tool selection after candidate compression on BFCL-V4 non-live and live multiple tasks, with detailed results in Appendix A. BFCL-live is closer to deployment

Setting	Method	Avg ms/query	Hit@1	Recall@10	Recall@50	MRR@10	nDCG@10	FullCoverage@10
Raw metadata	BM25	3909.4	.553	.571	.726	.625	.512	.413
Raw metadata	RAGAnything	821.6	.600	.568	.712	.663	.500	.393
Reported reference	SkillRouter†	–	.740	.704	–	.791	–	.520
Generated query only	BM25	**218.9**	.840	.880	.951	.881	.846	.760
Generated query only	RAGAnything	21505.9	.900	.922	.953	.925	**.882**	.827
Toollery	BM25	6332.0	**.920**	**.942**	**.968**	**.942**	.880	**.853**
Toollery	RAGAnything	1070.5	.587	.572	.719	.664	.510	.393

Table 1: SkillBench-compatible retrieval results, all values are reported as the average of the easy and hard splits. Toollery retrieves over metadata plus generated user-intent queries; Generated query only retrieves over retained generated queries without metadata. † denotes external reference quality values directly reported in the SkillRouter paper (Zheng et al., 2026), not a latency-controlled local rerun.

Setting	Method	Avg ms/query	Hit@1	Recall@10	Recall@50	MRR@10	nDCG@10	FullCoverage@10
Raw metadata	BM25	3.6	.643	.931	.976	.744	.790	.931
Raw metadata	RAGAnything	152.5	.737	.976	.995	.831	.867	.976
Generated query only	BM25	**0.3**	.468	.822	.934	.584	.642	.822
Generated query only	RAGAnything	81.8	.543	.855	.929	.653	.702	.855
Toollery	BM25	5.0	.731	.948	.986	.811	.845	.948
Toollery	RAGAnything	139.7	**.776**	**.980**	**.997**	**.856**	**.887**	**.980**

Table 2: Atomic skill/tool retrieval quality on BFCL-live multiple full-pool instances, where each original query is evaluated against the complete 457-tool candidate pool (Patil et al., 2024). Toollery retrieves over metadata plus generated user-intent queries; Generated query only retrieves over retained generated queries without metadata.

because the model selects from realistic live API schemas and all-candidate prompting becomes infeasible as the pool grows.

Tables 5 and 4 show that Toollery is most useful when the pool is large. At the complete 443-tool BFCL non-live scale, Qwen3-235B function calling improves from 79.13 to 94.00; at the complete 457-tool live scale, it improves from 81.13 to 88.63. This supports the candidate-compression view: reducing the action space stabilizes selection as libraries grow.

5 Discussion and Conclusion

Toollery is a candidate-compression layer for large skill libraries, not a replacement for trained function-calling models, universal routers, or generic RAG over tool documentation. It moves part of semantic matching offline by converting specifications into user-intent queries before final LLM decision making.

The main industry lesson is that generated queries are most useful when connected to original metadata: raw metadata can miss user phrasing, while generated-query-only retrieval can lose schema details. Backend choice remains a deployment ablation: BM25 is cheaper and strong on large skill pools, while RAGAnything is stronger on the BFCL-live atomic-tool pool.

Operationally, Toollery shifts work from online prompting to offline index maintenance. This cost is amortized for reused libraries, while rapidly changing registries should update only affected skills and monitor zero-query or low-confidence skills. At serving time, systems can start with a small candidate budget and expand k or fall back to broader retrieval when confidence is low.

Thus Toollery is complementary to existing routers and selectors: BM25 or RAG-style retrieval reduces the candidate set, while the final LLM still performs schema-aware selection and argument generation.

Setting	Method	Avg ms/query	Hit@1	Recall@10	Recall@50	MRR@10	nDCG@10	FullCoverage@10
Raw metadata	BM25	14.4	.780	.970	.995	.854	.883	.970
Raw metadata	RAGAnything	524.6	.825	**1.000**	**1.000**	.895	.922	**1.000**
Generated query only	BM25	**0.2**	.555	.885	.990	.668	.721	.885
Generated query only	RAGAnything	77.9	.605	.925	.985	.722	.772	.925
Toollery	BM25	25.9	.815	**1.000**	**1.000**	.887	.915	**1.000**
Toollery	RAGAnything	657.3	**.845**	**1.000**	**1.000**	**.904**	**.928**	**1.000**

Table 3: Atomic skill/tool retrieval quality on BFCL non-live multiple full-pool instances, where each original query is evaluated against the complete 443-tool candidate pool (Patil et al., 2024). Toollery retrieves over metadata plus generated user-intent queries; Generated query only retrieves over retained generated queries without metadata.

6 Limitations

Toollery depends on the coverage and precision of its generated queries. If the generator fails to produce realistic user intents for a skill, retrieval recall is bounded no matter which retrieval backend is used. Conversely, if generated queries are too broad, they can add noisy matches to the query-augmented document. The generated-query-only ablation uses source-skill recovery filtering to reduce this risk, but the filter does not provide a formal correctness guarantee.

The framework also adds offline maintenance cost. New or modified skills require query generation, representation, and index updates, and optional filtering adds further cost for generated-query-only retrieval. This cost is amortized across future requests, but rapidly changing ecosystems should track construction cost separately from online retrieval and final-selection latency.

Finally, Toollery addresses candidate selection, not full agent execution. It does not solve multi-step planning, tool sequencing, execution monitoring, recovery from failed calls, or argument correctness by itself. In multi-step agents, Toollery can reduce the candidate set at each decision point, but the broader planning policy and execution controller remain outside the scope of this work.

References

Mayank Agarwal, Ibrahim Abdelaziz, Kinjal Basu, Merve Unuvar, Luis A Lastras, Yara Rizk, and Pavan Kapanipathi. 2025. Toolrm: Outcome reward models for tool-calling large language models. *arXiv preprint arXiv:2509.11963*.

Amazon Web Services. 2024. Boto3: The AWS SDK for Python. https://boto3.amazonaws.com/v1/documentation/api/latest/index.html. Accessed: 2024-05-01.

Tiantian Gan and Qiyao Sun. 2025. Rag-mcp: Mitigating prompt bloat in llm tool selection via retrieval-augmented generation. *arXiv preprint arXiv:2505.03275*.

Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. API-bank: A comprehensive benchmark for tool-augmented LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3102–3116, Singapore. Association for Computational Linguistics.

Jianghao Lin, Xinyuan Wang, Xinyi Dai, Menghui Zhu, Bo Chen, Ruiming Tang, Yong Yu, and Weinan Zhang. 2025. Masstool: A multi-task search-based tool retrieval framework for large language models. *arXiv preprint arXiv:2507.00487*.

Guozhao Mo, Wenliang Zhong, Jiawei Chen, Xuanang Chen, Yaojie Lu, Hongyu Lin, Ben He, Xianpei Han, and Le Sun. 2025. Livemcpbench: Can agents navigate an ocean of mcp tools? *arXiv preprint arXiv:2508.01780*.

Rodrigo Nogueira, Wei Yang, Jimmy Lin, and Kyunghyun Cho. 2019. Document expansion by query prediction. *arXiv preprint arXiv:1904.08375*.

Varatheepan Paramanayakam, Andreas Karatzas, Iraklis Anagnostopoulos, and Dimitrios Stamoulis. 2025. Less is more: Optimizing function calling for llm execution on edge devices. In *2025 Design, Automation & Test in Europe Conference (DATE)*, pages 1–7. IEEE.

Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E Gonzalez. 2025. The berkeley function calling leaderboard (bfcl): From tool use to agentic evaluation of large language models. In *Forty-second International Conference on Machine Learning*.

Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. 2024. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems*, 37:126544–126565.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, and 1 others. 2023. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves

to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551.

Jun Wang, Jiamu Zhou, Xihuai Wang, Xiaoyun Mo, Haoyu Zhang, Qiqiang Lin, Jincheng Jincheng, Muning Wen, Weinan Zhang, and Qiuying Peng. 2025. Hammerbench: Fine-grained function-calling evaluation in real mobile assistant scenarios. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 3350–3376.

Zhiyong Wu, Chengcheng Han, Zichen Ding, Zhenmin Weng, Zhoumianze Liu, Shunyu Yao, Tao Yu, and Lingpeng Kong. 2024. Os-copilot: Towards generalist computer agents with self-improvement. *arXiv preprint arXiv:2402.07456*.

Hanwen Xu, Xuyao Huang, Yuzhe Liu, Kai Yu, and Zhijie Deng. 2025. Tps-bench: Evaluating ai agents' tool planning\& scheduling abilities in compounding tasks. *arXiv preprint arXiv:2511.01527*.

Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35:20744–20757.

Hanjie Zheng, Chenglin Li, Yibing Liu, Jiazhan Feng, Wenhao Wu, Jiabao Guo, Yicheng Peng, Tianhang Li, Qian Zhang, Chi Zhang, and 1 others. 2026. Skillrouter: Retrieve-and-rerank skill selection for llm agents at scale. *arXiv preprint arXiv:2603.22455*.

A Additional End-to-End Results

Tables 4 and 5 report BFCL-V4 live and non-live multiple end-to-end selection accuracy under increasing candidate-pool sizes. The original non-live multiple split contains 200 requests and 443 unique tools, while the live multiple split contains 1,053 requests and 457 unique tools. The final column is therefore the complete candidate pool for each split, not a 1,000-candidate setting. The columns compare final selection when the model sees all candidates in the sampled pool against selection after Toollery retrieves a compact subset. We include both prompt-based selection and function-calling selection to test whether candidate compression helps across different final decision interfaces.

B Reproducibility and Deployment Details

B.1 Offline Construction Cost

To make the offline cost explicit, we measure query generation and recovery filtering on the SkillBench easy and hard splits using DeepSeek-V4-Pro as both generator and verifier. Each skill receives three generated candidate queries and each non-empty candidate is checked against the source skill plus eight random distractors. Table 7 reports the measured offline construction cost for the full 78,361-skill easy pool and 79,141-skill hard pool. Under official DeepSeek-V4-Pro cache-miss input and output pricing,[2] this corresponds to approximately $0.22 per 100 skills on average. For the easy split, construction produces 235K generated candidates, 228K retained queries, 361.8M total tokens, and approximately $168.70; for the hard split, the corresponding values are 237K candidates, 230K retained queries, 365.4M tokens, and $170.38. These values describe offline construction only; the online latency tables use warmed indexes and exclude this one-time generation and filtering cost.

[2] https://api-docs.deepseek.com/quick_start/pricing/, accessed June 2, 2026.

Table 4: BFCL-V4 live multiple results

Model	$\leq$ 10	20	30	40	50	100	457
Qwen3-4B (Prompt)	85.50	86.00	88.00	82.50	78.00	69.50	24.88
Qwen3-4B (Prompt) + Toollery	87.00 ⇑	84.50	87.00	86.50 ⇑	82.50 ⇑	81.00 ⇑	83.75 ⇑
Qwen3-8B (Prompt)	97.00	93.50	92.00	84.00	83.00	84.50	38.50
Qwen3-8B (Prompt) + Toollery	95.50	**98.50** ⇑	**98.50** ⇑	92.50 ⇑	**94.50**	86.50 ⇑	79.38 ⇑
DeepSeek V3.2 (Prompt)	94.50	80.00	84.00	89.50	79.50	63.00	77.00
DeepSeek V3.2 (Prompt) + Toollery	98.00 ⇑	78.50	94.50 ⇑	**99.50** ⇑	89.50 ⇑	89.00 ⇑	83.13 ⇑
GPT-5-mini (Prompt)	83.00	93.00	90.00	87.00	88.00	88.00	79.75
GPT-5-mini (Prompt) + Toollery	83.50 ⇑	94.00 ⇑	89.50	95.00 ⇑	89.50 ⇑	89.00 ⇑	84.50 ⇑
GPT-5-mini (FC)	67.00	62.00	63.50	62.00	61.00	63.50	57.00
GPT-5-mini (FC) + Toollery	74.50 ⇑	67.50 ⇑	69.00 ⇑	69.50 ⇑	67.00 ⇑	67.50 ⇑	66.00 ⇑
Qwen3-8B (FC)	93.50	94.50	92.50	83.00	83.50	83.00	60.00
Qwen3-8B (FC) + Toollery	93.50	98.00 ⇑	90.50	99.00 ⇑	91.50 ⇑	88.50 ⇑	85.88 ⇑
Qwen3-235B (FC)	98.00	93.00	87.50	89.50	85.50	80.00	81.13
Qwen3-235B (FC) + Toollery	**98.50** ⇑	92.50	89.00 ⇑	89.50	90.00	**91.50** ⇑	**88.63** ⇑

Table 5: BFCL-V4 non-live multiple results

Model	$\leq$ 10	20	30	40	50	100	443
Qwen3-4B (Prompt)	95.50	99.50	97.50	96.50	95.50	90.00	24.25
Qwen3-4B (Prompt) + Toollery	**100.00** ⇑	97.50	98.00 ⇑	95.00	98.00 ⇑	91.50 ⇑	83.63 ⇑
Qwen3-8B (Prompt)	97.50	98.50	**99.50**	97.50	97.00	89.50	37.63
Qwen3-8B (Prompt) + Toollery	99.50 ⇑	**100.00** ⇑	97.50	96.00	98.00 ⇑	91.00 ⇑	80.63 ⇑
DeepSeek V3.2 (Prompt)	86.00	99.50	97.00	94.50	96.50	85.50	70.25
DeepSeek V3.2 (Prompt) + Toollery	97.50 ⇑	97.50	95.00	98.00 ⇑	**98.50** ⇑	89.50 ⇑	81.38 ⇑
GPT-5-mini (Prompt)	88.00	87.50	92.50	85.00	88.50	88.00	80.00
GPT-5-mini (Prompt) + Toollery	88.00	89.50 ⇑	91.50	86.50 ⇑	95.00 ⇑	90.00 ⇑	82.00 ⇑
GPT-5-mini (FC)	79.50	69.50	65.50	72.00	70.50	63.50	58.63
GPT-5-mini (FC) + Toollery	81.50 ⇑	77.00 ⇑	75.50 ⇑	73.00 ⇑	76.00 ⇑	69.50 ⇑	68.63 ⇑
Qwen3-8B (FC)	97.50	97.50	97.50	**98.50**	96.00	85.00	60.75
Qwen3-8B (FC) + Toollery	99.50 ⇑	96.50	97.50	96.50	97.00 ⇑	90.50 ⇑	86.75 ⇑
Qwen3-235B (FC)	95.50	99.50	96.50	96.00	97.50	80.00	79.13
Qwen3-235B (FC) + Toollery	96.00 ⇑	99.00	98.50 ⇑	98.00 ⇑	98.00 ⇑	**93.50** ⇑	**94.00** ⇑

Item	Value used in reported runs
SkillBench pools	78,361 easy skills; 79,141 hard skills; 75 requests per split
BFCL-live end-to-end pool	457 tools; 1,053 live multiple requests; final column uses the complete pool
BFCL-non-live end-to-end pool	443 tools; 200 multiple requests; final column uses the complete pool
Generated-query prompt	asks the LLM to generate realistic proxy user requests from skill/tool name, description, schema, and up to 3 labeled example requests when available; output is a JSON array with query, scenario type, and notes
Generated-query decoding	DeepSeek-V4-Pro through an OpenAI-compatible API; 3 generated queries per skill/tool; seed 31; 4 workers; batch size 4
Verifier prompt	asks DeepSeek-V4-Pro to choose one matching skill/tool id from the target plus distractors, or NONE if no candidate fits
Distractor construction	verifier pool uses the target plus 8 randomly sampled non-gold distractors; scaled candidate pools sample the gold tool plus non-gold tools without replacement
BM25 configuration	local tokenizer in `toollery.text`; Okapi BM25 with $k_1 = 1.5, b = 0.75$; query-augmented documents concatenate name, category, description, schema, and generated queries
RAGAnything configuration	text-only RAGAnything/LightRAG cache with one compact chunk per skill/tool; retrieval queries LightRAG chunk vectors and recovers tool ids from returned context order
Retrieval embedding model	Qwen/Qwen3-Embedding-0.6B for neural retrieval and latency replay
Aggregation parameters	top-$c = 3$ averaged query scores with evidence bonus $b = 5, \lambda = 0.05$
Candidate budgets	top-50 for retrieval-quality tables; BFCL-live efficiency diagnostics use top-3 retrieved candidates; end-to-end table columns denote the all-candidate pool size before Toollery compression
Latency protocol	warmed indexes/caches; reports online retrieval or final-selection latency only; excludes offline query generation, verification, embedding, and index construction
Offline-cost sample	100 easy-pool skills; DeepSeek-V4-Pro generator and verifier; 3 generated queries per skill; target plus 8 random distractors for verification; cost computed with cache-miss input pricing
Release plan	prompts, normalized generated-query artifacts, run configurations, and metric scripts will be released with the anonymized artifact

Table 6: Reproducibility details for the main retrieval and efficiency results. Secrets and local API endpoints are omitted; paths in the released run configs identify the exact output artifacts.

Setting	Skills	Cand.	Ret.	Tok.	Cost
SkillBench easy	78,361	235K	228K	361.8M	$168.70
SkillBench hard	79,141	237K	230K	365.4M	$170.38

Table 7: Offline construction cost for generated-query construction and verifier filtering. The measured rows use the SkillBench easy and hard splits with three generated queries per skill. Cost uses official DeepSeek-V4-Pro pricing: $0.435/M cache-miss input tokens and $0.87/M output tokens; cache-hit discounts are not applied.